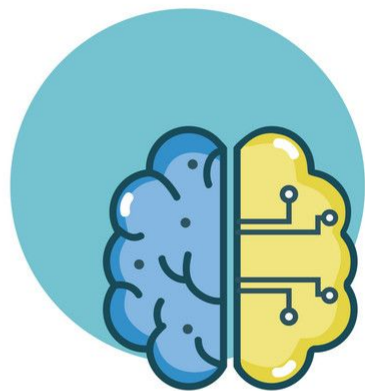


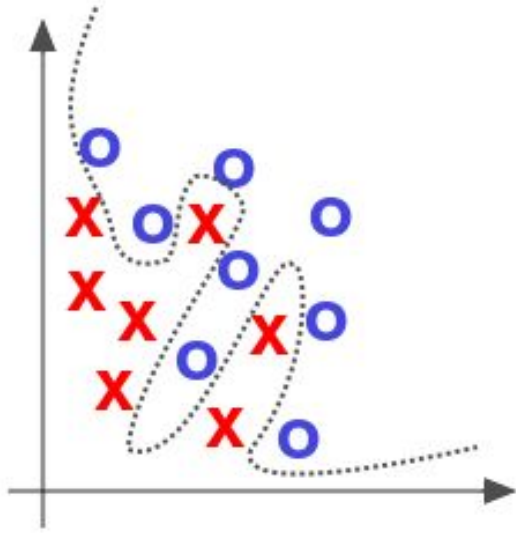
# INTRODUCTION TO MACHINE LEARNING

## REGULARIZATION



Elisa Ricci

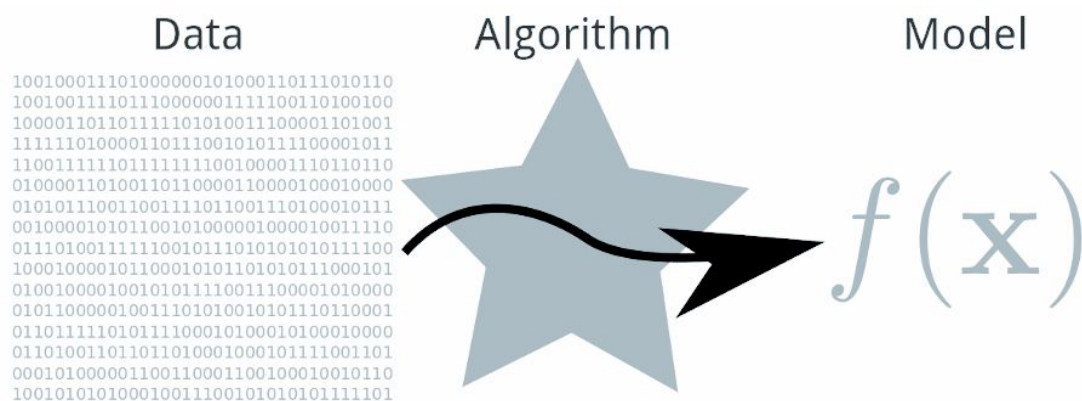




AVOID OVERFITTING

# MACHINE LEARNING IDEA

- ML allows computers to acquire knowledge.
- Knowledge is acquired through **algorithms** by learning and inferring from **data**.
- Knowledge is represented by a **model**.
- The model is used on future data.



# LINEAR MODELS

- Perceptron algorithm is one example of a linear classifier
- Many, many other algorithms learn a line (i.e. a setting of a linear combination of weights)
- Goals:
  - Explore a number of linear training algorithms

# MODEL-BASED MACHINE LEARNING

1. Pick a **model**
  - e.g. a hyperplane, a decision tree,...
  - A model is defined by a collection of parameters
2. Pick a criterion to optimize (aka **objective function**)
  - e.g. training error
3. Develop a **learning algorithm**
  - the algorithm should try and minimize the criteria, sometimes in a heuristic way (i.e. non-optimally), sometimes exactly

# MODEL-BASED MACHINE LEARNING

1. Pick a model (a model is a collection of parameters)

$$0 = b + \sum_{j=1}^m w_j f_j$$

2. Pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

3. Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n 1[y_i(w \cdot x_i + b) \leq 0]$$

Find  $w$  and  $b$  that  
minimize the 0/1 loss  
(i.e. training error)

# MODEL-BASED MACHINE LEARNING

1. Pick a model (a model is a collection of parameters)

$$0 = b + \sum_{j=1}^m w_j f_j$$

2. Pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n \exp(-y_i (w \cdot x_i + b))$$

use a convex surrogate  
loss function

3. Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i (w \cdot x_i + b))$$

Find  $w$  and  $b$  that  
minimize the  
surrogate loss

# GRADIENT DESCENT

- Pick a starting point ( $w$ )
- Repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \frac{d}{dw_j} loss$$

Why negative?





# GRADIENT DESCENT

For our choice of the loss we have:

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}(w)$$

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

# PERCEPTRON LEARNING ALGORITHM!

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

**if**  $\text{prediction} * \text{label} \leq 0$ : // they don't agree

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

**Note:** for gradient descent, we always update

$$b = b + \text{label}$$

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b))$$

In practice

$$w_j = w_j + x_{ij} y_i c$$

where

$$c = \eta \exp(-y_i (w \cdot x_i + b))$$

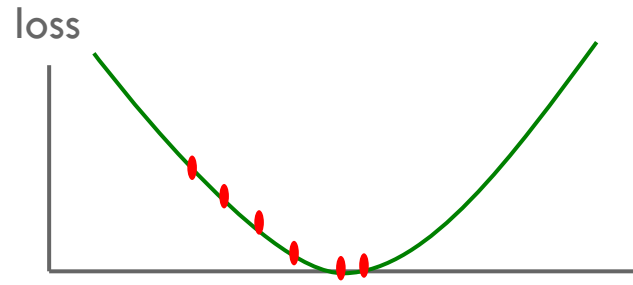
# ONE CONCERN

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b))$$

We are calculating this on the **training set**

We still need to be careful about **overfitting!**

The  $\min_{w,b} \text{Loss}$  on the training set is generally NOT the min for the test set



How did we deal with this?

# REGULARIZATION

- A **regularizer** is an additional criterion to the loss function to make sure that we do not overfit
- It is called a regularizer since it tries to keep the parameters more normal/regular
- It is a bias on the model that forces the learning to prefer certain types of weights over others

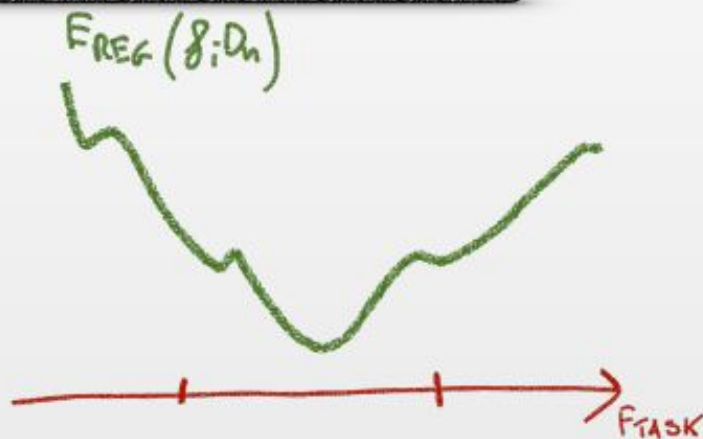
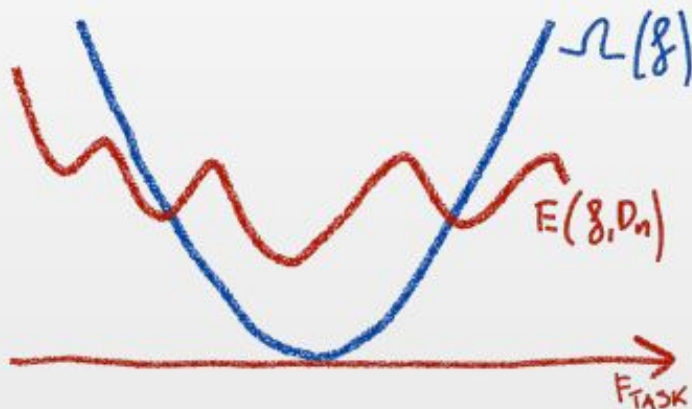
$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w,b)$$

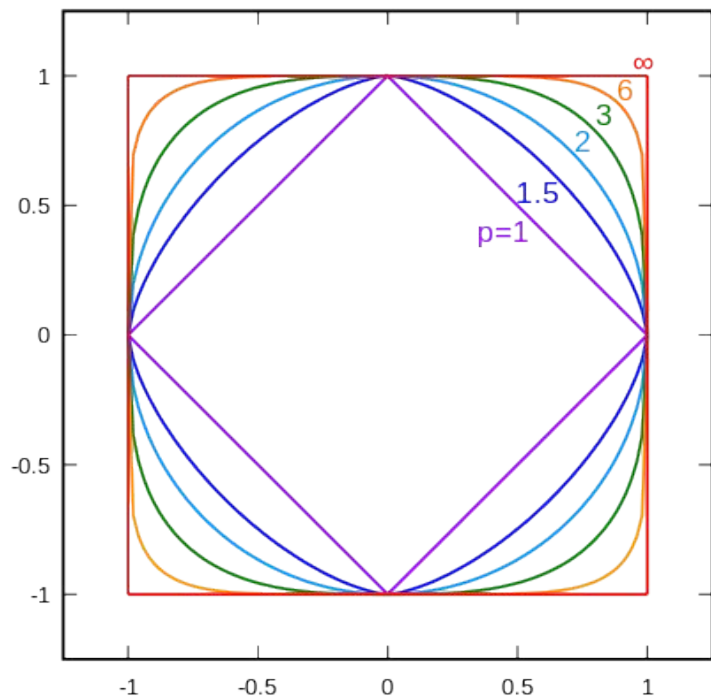
# SO FAR...

Modification of the training error function with a term  $\Omega(f)$  that typically penalizes complex solutions

$$E_{\text{reg}}(f; \mathcal{D}_n) = E(f; \mathcal{D}_n) + \lambda_n \Omega(f)$$

TRADE-OFF  
PARAMETER





# REGULARIZERS

# REGULARIZATION

- A **regularizer** is an additional criterion to the loss function to make sure that we do not overfit
- It is called a regularizer since it tries to keep the parameters more normal/regular
- It is a bias on the model that forces the learning to prefer certain types of weights over others

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(y y') + \lambda \operatorname{regularizer}(w, b)$$

# REGULARIZERS

$$0 = b + \sum_{j=1}^n w_j f_j$$

Should we allow all possible weights?

Any preferences?

What makes for a “simpler” model for a linear model?



# REGULARIZERS: PRINCIPLES

$$0 = b + \sum_{j=1}^n w_j f_j$$

- Generally, we do not want huge weights: if weights are large, a small change in a feature can result in a large change in the prediction
- Might also prefer weights of 0 for features that are not useful

# REGULARIZERS

$$0 = b + \sum_{j=1}^n w_j f_j$$

How do we encourage small weights? or penalize large weights?

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(y_i y'_i) + \lambda \operatorname{regularizer}(w, b)$$

# COMMON REGULARIZERS

sum of the weights

$$r(w, b) = \sum |w_j|$$

sum of the squared weights

$$r(w, b) = \sqrt{\sum |w_j|^2}$$

What's the difference between these?

# COMMON REGULARIZERS

sum of the weights

$$r(w, b) = \sum |w_j|$$

sum of the squared weights

$$r(w, b) = \sqrt{\sum |w_j|^2}$$

Squared weights penalizes large values more  
Sum of weights will penalize small values more

# P-NORM

sum of the weights (1-norm)

$$r(w, b) = \sum |w_j|$$

sum of the squared weights  
(2-norm)

$$r(w, b) = \sqrt{\sum |w_j|^2}$$

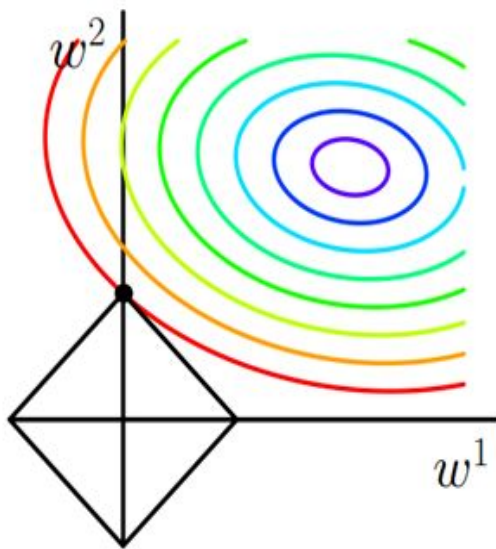
---

p-norm

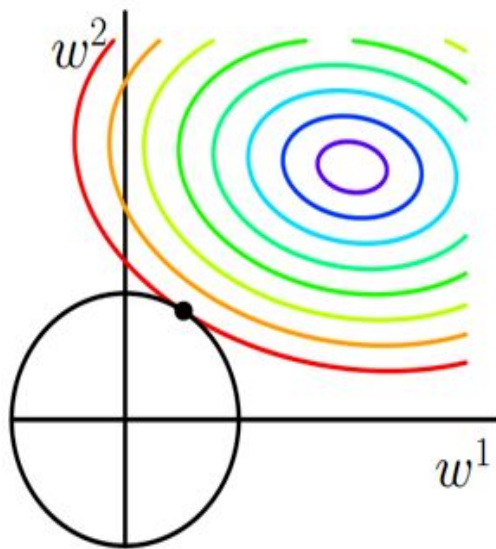
$$r(w, b) = \sqrt[p]{\sum |w_j|^p} = \|w\|^p$$

Smaller values of  $p$  ( $p < 2$ ) encourage sparser vectors  
Larger values of  $p$  discourage large weights more

# L1/L2-NORMS VISUALIZED

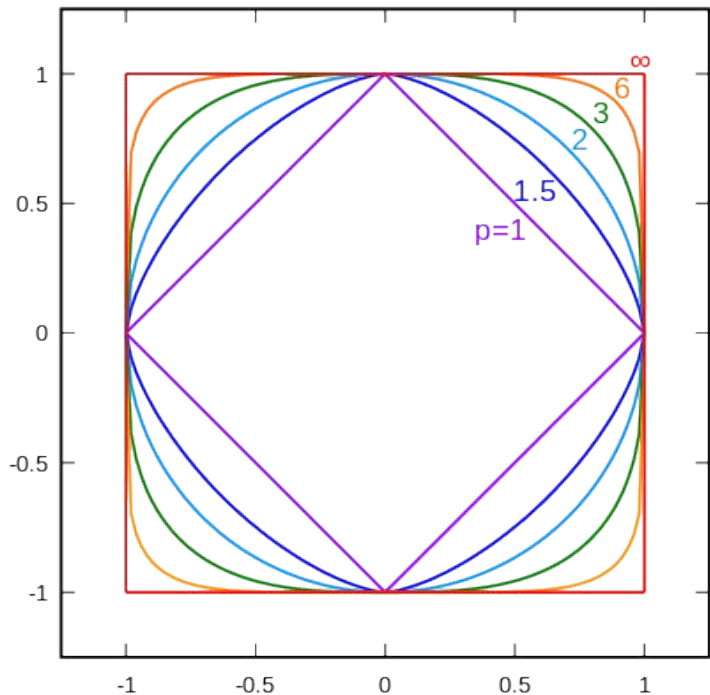


(a)  $\ell_1$ -ball meets quadratic function.  
 $\ell_1$ -ball has corners. It's very likely that the meet-point is at one of the corners.



(b)  $\ell_2$ -ball meets quadratic function.  
 $\ell_2$ -ball has no corner. It is very unlikely that the meet-point is on any of axes."

# P-NORMS VISUALIZED



all  $p$ -norms penalize larger weights

$p < 2$  tends to create sparse (i.e. lots of 0 weights)

$p > 2$  tends to like similar weights

# MODEL-BASED MACHINE LEARNING

1. Pick a model

$$0 = b + \sum_{j=1}^n w_j f_j$$

2. Pick a criteria to optimize (aka objective function)

$$\sum_{i=1}^n \text{loss}(yy') + \lambda \text{regularizer}(w)$$

3. Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \text{loss}(yy') + \lambda \text{regularizer}(w)$$

Find  $w$  and  $b$   
that minimize



# MINIMIZING WITH A REGULARIZER

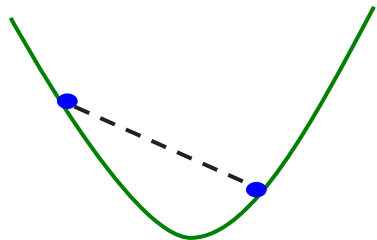
We know how to solve convex minimization problems using gradient descent:

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy')$$

If we can ensure that the loss + regularizer is convex then we could still use gradient descent:

$$\operatorname{argmin}_{w,b} \underbrace{\sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w)}_{\text{make convex}}$$

# CONVEXITY



One definition: The line segment between any two points on the function is *above* the function

Mathematically,  $f$  is convex if for all  $x_1, x_2$ :

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \forall \quad 0 < t < 1$$

the value of the function at  
some point between  $x_1$  and  $x_2$

the value at some point on the **line segment** between  $x_1$  and  $x_2$

# MINIMIZING WITH A REGULARIZER

We know how to solve convex minimization problems using gradient descent:

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \operatorname{loss}(yy')$$

If we can ensure that the loss + regularizer is convex then we could still use gradient descent:

$$\operatorname{argmin}_{w,b} \underbrace{\sum_{i=1}^n \operatorname{loss}(yy') + \lambda \operatorname{regularizer}(w)}$$

convex as long as both loss and regularizer are convex

# P-NORMS ARE CONVEX

$$r(w, b) = \sqrt[p]{\sum |w_j|^p} = \|w\|^p$$

p-norms are convex for  $p \geq 1$

# MODEL-BASED MACHINE LEARNING

1. Pick a model

$$0 = b + \sum_{j=1}^n w_j f_j$$

2. Pick a criteria to optimize (aka objective function)

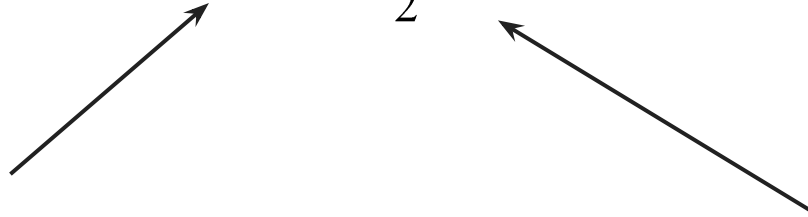
$$\sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

3. Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

Find  $w$  and  $b$   
that minimize

# OUR OPTIMIZATION CRITERION

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$


**Loss function:** penalizes examples where the prediction is different than the label

**Regularizer:** penalizes large weights

Key: this function is convex allowing us to use gradient descent

# GRADIENT DESCENT

- Pick a starting point ( $w$ )
- Repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} (\text{loss}(w) + \text{regularizer}(w, b))$$

---

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

# SOME MORE MATHS

$$\frac{d}{dw_j} \text{objective} = \frac{d}{dw_j} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

⋮

(some math happens)

$$= - \sum_{i=1}^n y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) + \lambda w_j$$



# GRADIENT DESCENT

- Pick a starting point ( $w$ )
- Repeat until loss doesn't decrease in any dimension:
  - pick a dimension
  - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} (\text{loss}(w) + \text{regularizer}(w, b))$$

---

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i (w \cdot x_i + b)) - \eta \lambda w_j$$

# THE UPDATE

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b)) - \eta \lambda w_j$$

learning rate

direction to update

constant: how far from wrong

regularization

The diagram illustrates the components of the weight update equation. Arrows point from the following labels to their corresponding terms in the equation: 'learning rate' points to  $\eta$ , 'direction to update' points to  $y_i x_{ij}$ , 'constant: how far from wrong' points to  $\exp(-y_i (w \cdot x_i + b))$ , and 'regularization' points to  $\eta \lambda w_j$ . The term  $\eta \lambda w_j$  is highlighted with a light blue background.

What effect does the regularizer have?

# THE UPDATE

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b)) - \eta \lambda w_j$$

learning rate      direction to update      constant: how far from wrong      regularization

If  $w_j$  is positive, reduces  $w_j$   
If  $w_j$  is negative, increases  $w_j$  } moves  $w_j$  towards 0

# L1 REGULARIZATION

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \|w\|$$

---

$$\frac{d}{dw_j} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \lambda \|w\|$$

$$= -\sum_{i=1}^n y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) + \lambda \operatorname{sign}(w_j)$$

# THE UPDATE

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i (w \cdot x_i + b)) - \eta \lambda \text{sign}(w_j)$$

learning rate

direction to  
update

constant: how far from wrong

regularization

If  $w_j$  is positive, reduces by a constant  
If  $w_j$  is negative, increases by a constant

} moves  $w_j$  towards 0  
**regardless of magnitude**

# REGULARIZATION WITH P-NORMS

**L1:**

$$w_j = w_j + \eta(loss\_correction - \lambda sign(w_j))$$

**L2:**

$$w_j = w_j + \eta(loss\_correction - \lambda w_j)$$

**Lp:**

$$w_j = w_j + \eta(loss\_correction - \lambda c w_j^{p-1})$$

# MODEL-BASED MACHINE LEARNING

Develop a learning algorithm

$$\operatorname{argmin}_{w,b} \sum_{i=1}^n \exp(-y_i(w \cdot x_i + b)) + \frac{\lambda}{2} \|w\|^2$$

Is gradient descent the only way to find  $w$  and  $b$ ?

No! Many other ways to find the minimum.

Some don't even require iteration

Whole field called convex optimization

# REGULARIZERS SUMMARIZED

- L1 is popular because it tends to result in sparse solutions (i.e. lots of zero weights). However, it is not differentiable, so it only works for gradient descent solvers
- L2 is also popular because for some loss functions, it can be solved directly (no gradient descent required, though often iterative solvers still)
- Lp is less popular since they don't tend to shrink the weights enough



# THE OTHER LOSS FUNCTIONS

Without regularization, the generic update is:

$$w_j = w_j + \eta y_i x_{ij} c$$

where

$$c = \exp(-y_i(w \cdot x_i + b))$$

exponential

$$c = 1[y y' < 1]$$

hinge loss

---

$$w_j = w_j + \eta (y_i - (w \cdot x_i + b)) x_{ij}$$

squared error

# MANY METHODS

- (Ordinary) Least squares: squared loss
- Ridge regression: squared loss with L2 regularization
- Lasso regression: squared loss with L1 regularization
- Elastic regression: squared loss with L1 AND L2 regularization
- Logistic regression: logistic loss

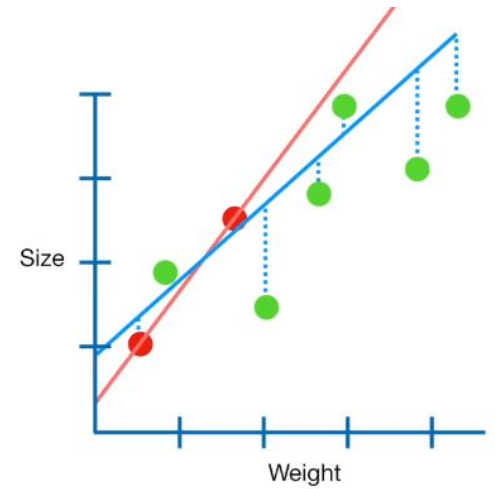
# REGRESSION

Squared loss with L2 regularization

$$L(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \mathbf{w})^2 + \lambda \sum_{j=1}^p w_j^2$$

Lasso regression

$$L(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \mathbf{w})^2 + \lambda \sum_{j=1}^p |w_j|$$



[Link](#)

# LOGISTIC REGRESSION

**Log loss** or **binary cross-entropy loss**

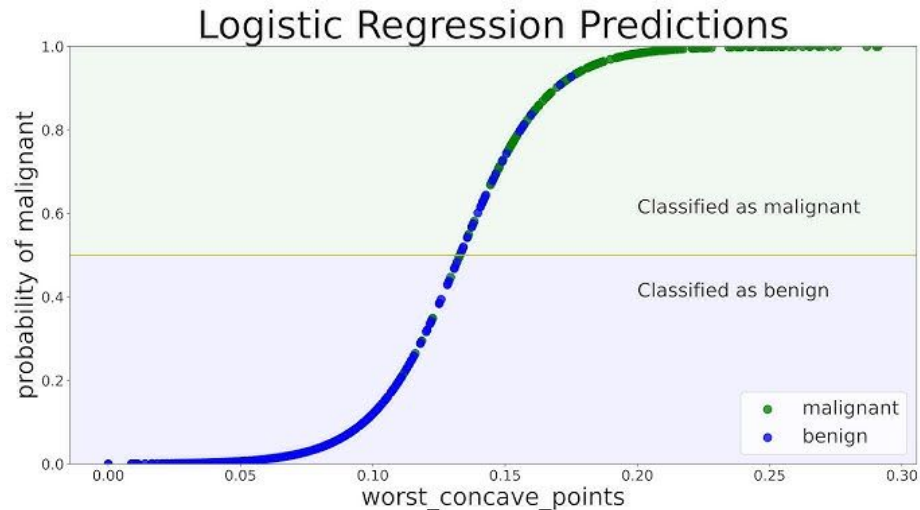
$$L(\mathbf{w}) = - \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where.

$$\hat{y}_i = \sigma(\mathbf{x}_i^T \mathbf{w})$$

and:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



# QUESTIONS?



Some slides are taken from David Kauchak